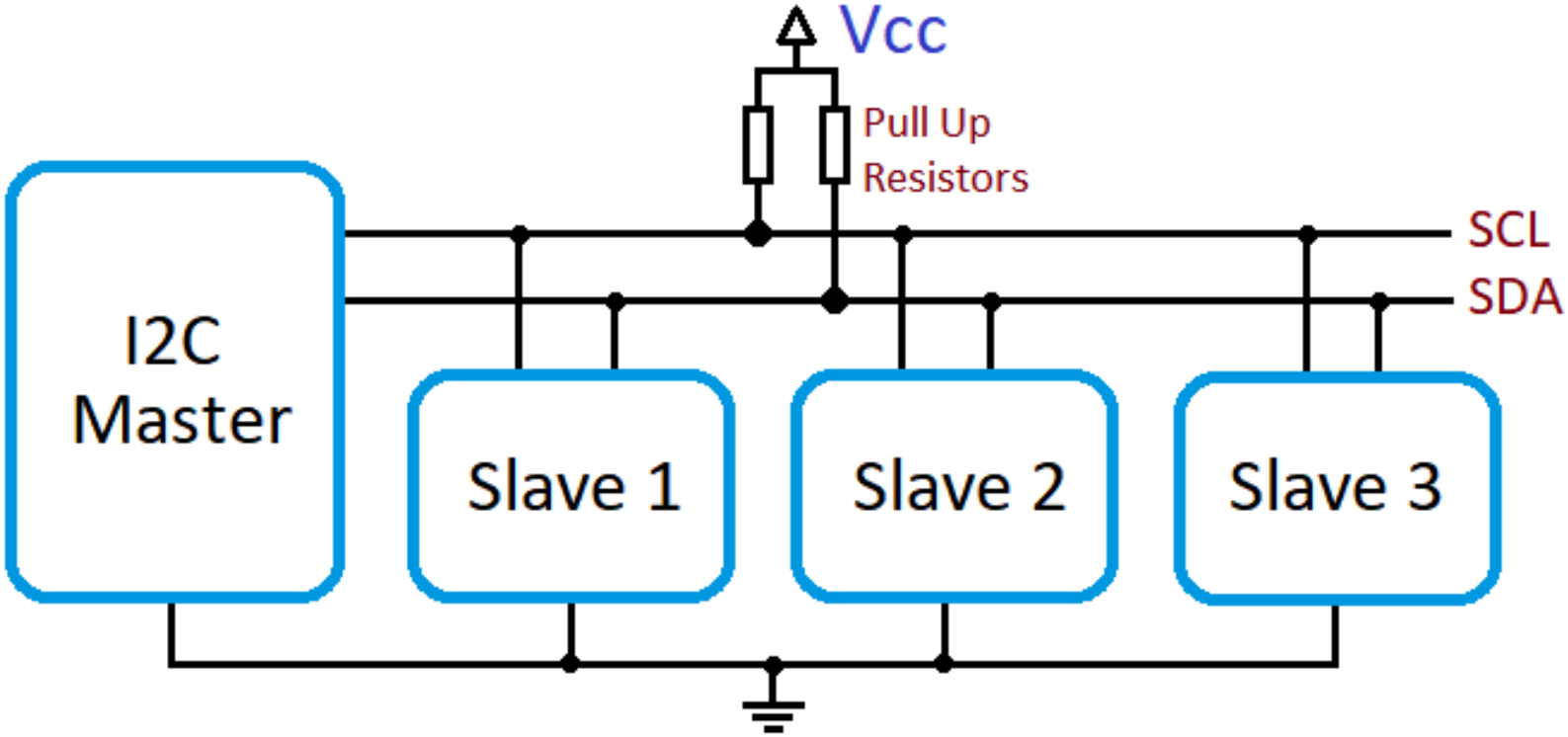
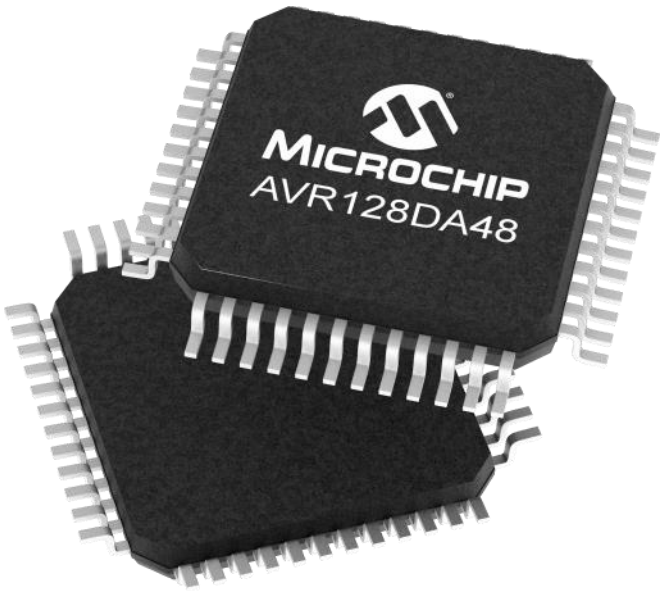
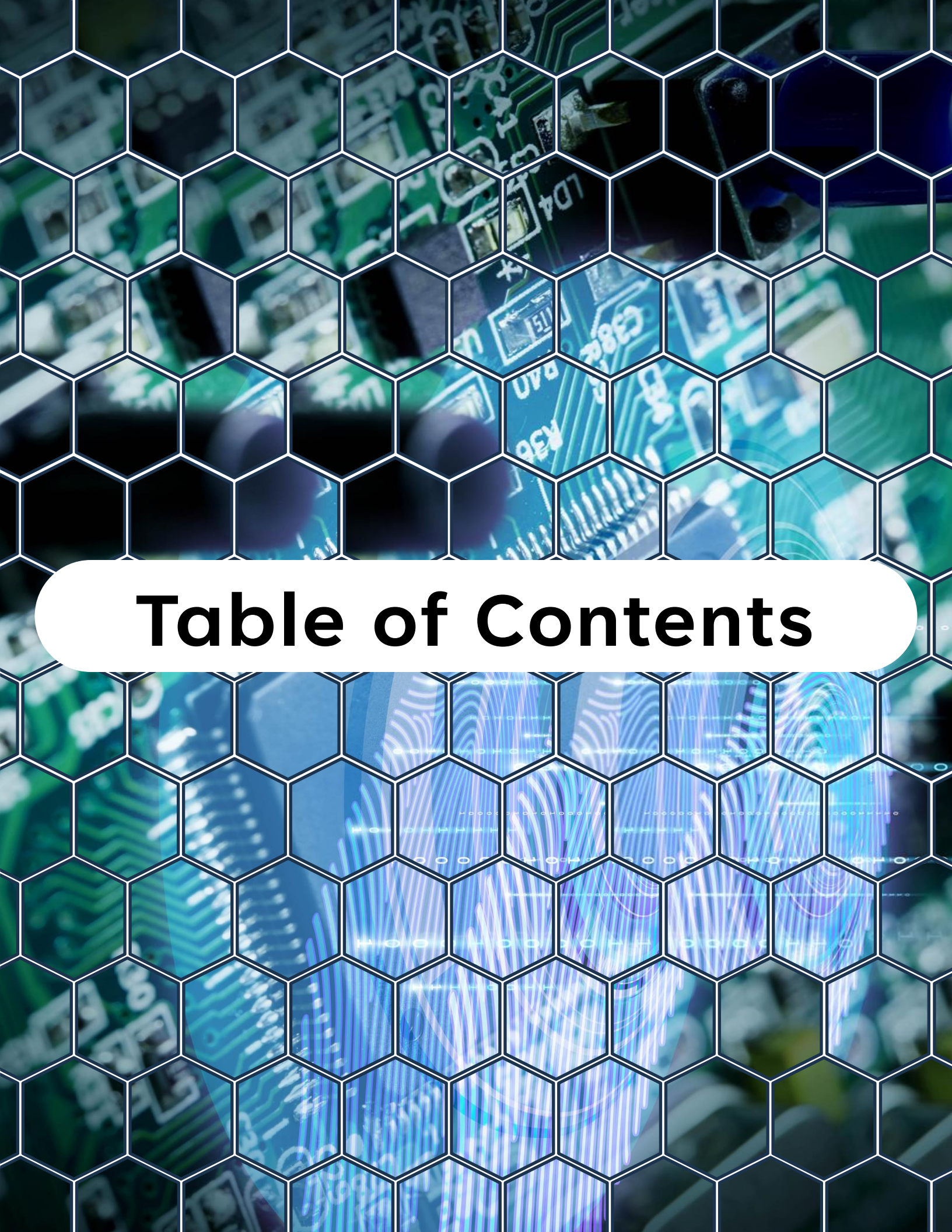
# “Bare-Metal Embedded Systems with AVR128DA48 Course”

## Day 16 - I2C Architecture and Interrupt-Driven TWI

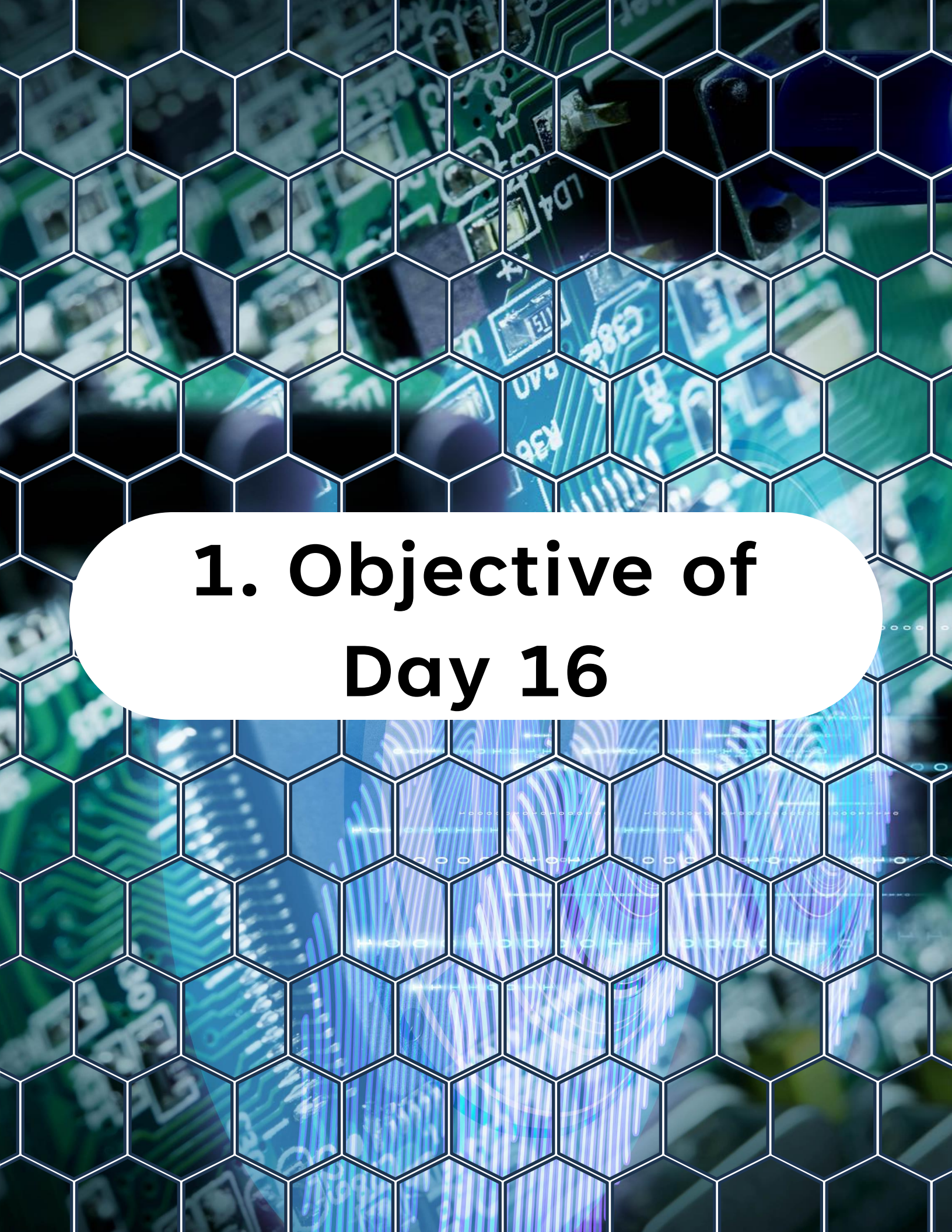




# Table of Contents

# Table of Contents



# 1. Objective of Day 16

# 1. Objective of Day 16

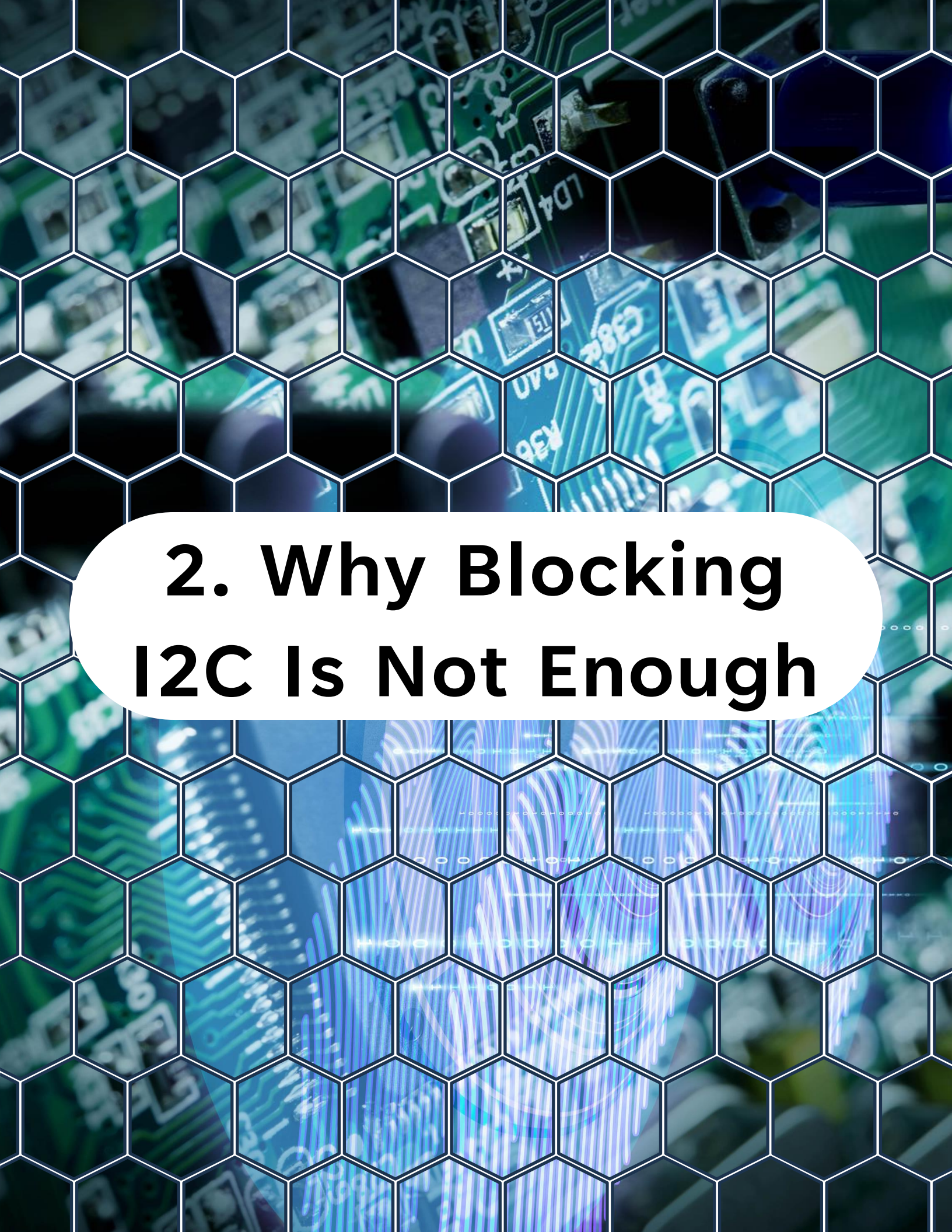
Yesterday you made I2C work.

Today you make it scalable.

You will move from simple blocking transactions to:

- A structured I2C driver architecture
- Interrupt-driven transfers
- State-machine-based control
- Multi-byte read/write handling
- Timeout and error management

By the end of this day, your TWI implementation will look like production firmware, not a lab demo.



# 2. Why Blocking I2C Is Not Enough

# 2. Why Blocking I2C Is Not Enough

Blocking code like this:

```
while (!(TWI0.MSTATUS & TWI_WIF_bm));
```

is fine for:

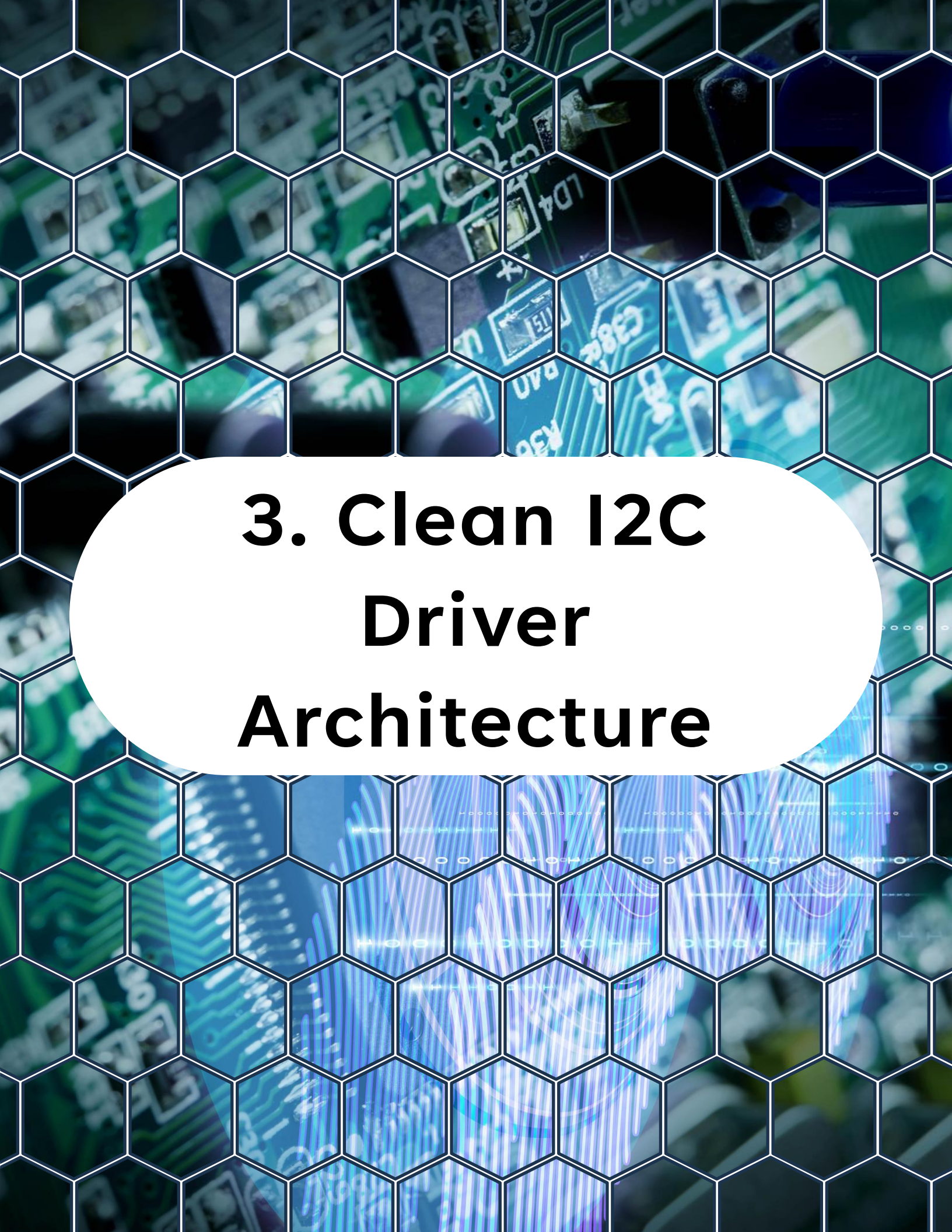
- Short demos
- Single-device systems
- No timing constraints

But it fails when:

- You need long transfers
- You must keep the main loop responsive
- You need cooperative multitasking
- You want deterministic timing

Blocking I2C scales poorly.

Interrupt-driven I2C solves this.



# 3. Clean I2C Driver Architecture

# 3. Clean I2C Driver Architecture

A proper TWI driver should have layers:

**Layer 1 - Hardware Control**

- Register access
- Interrupt enable/disable
- START, STOP, ACK commands

**Layer 2 - Transaction Engine**

- State machine
- Buffer management
- Error handling

**Layer 3 - Device Driver**

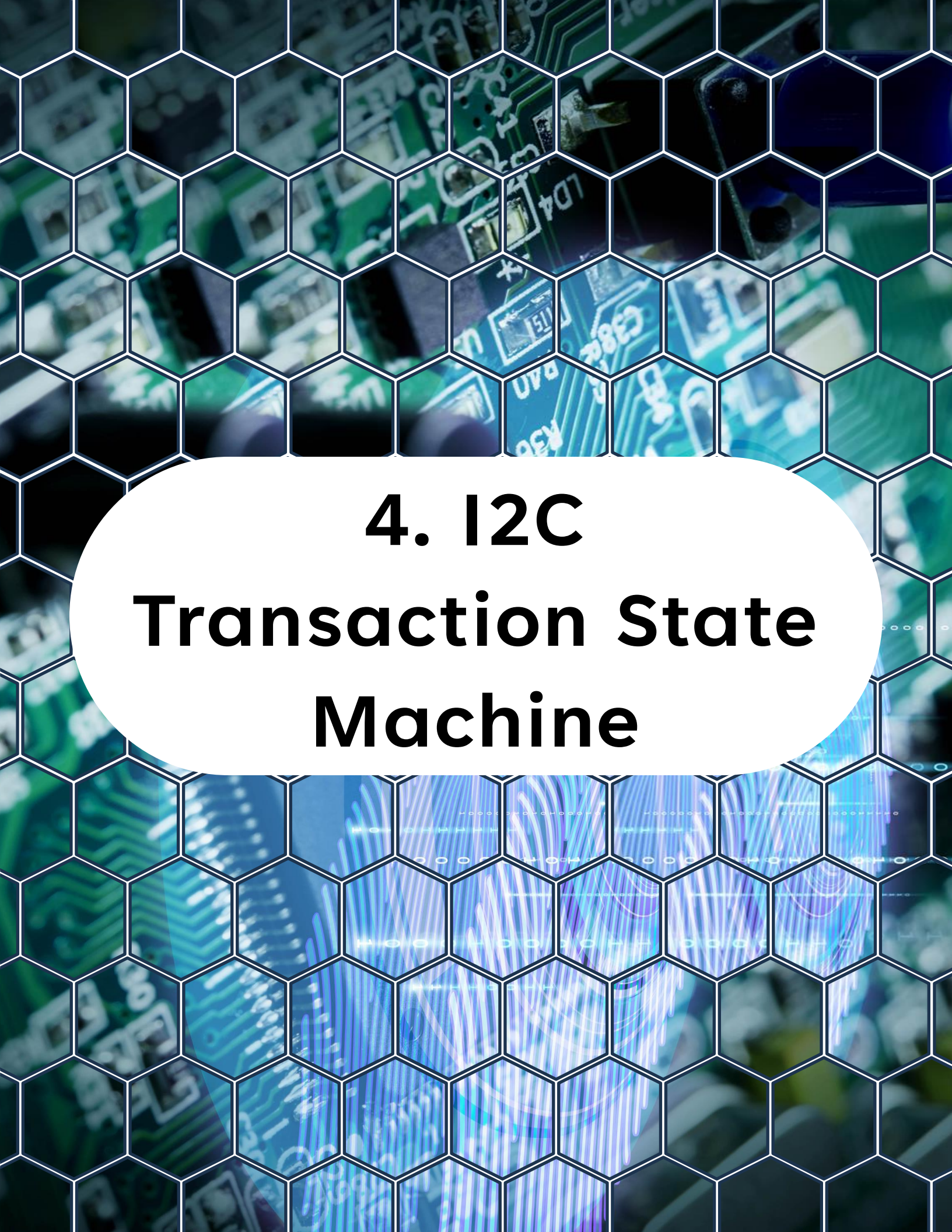
- sensor_read_reg()
- eeprom_write_page()
- rtc_read_time()

# 3. Clean I2C Driver Architecture

**Layer 4 – Application**

- Uses device APIs only

Never let application code touch TWI registers directly.



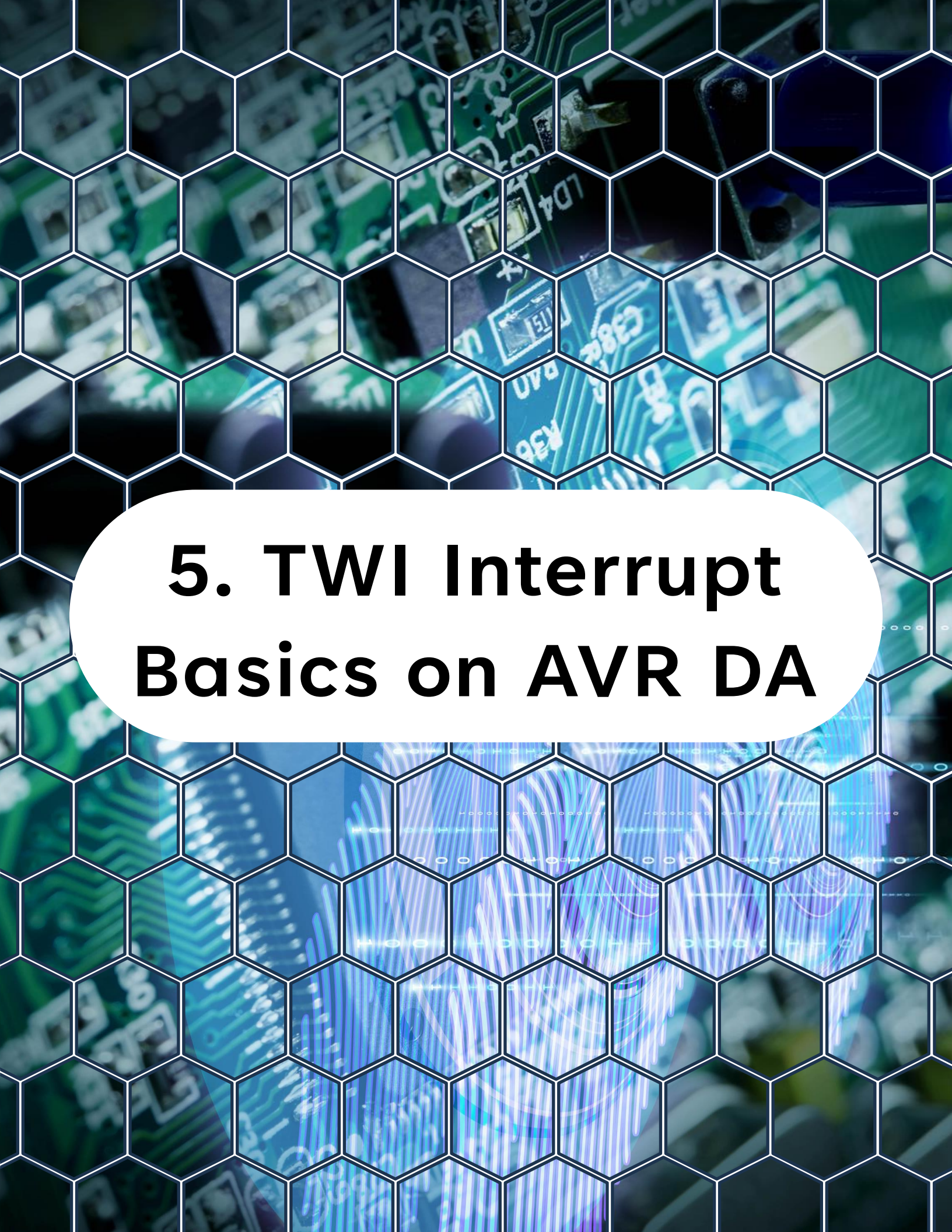
# 4. I2C Transaction State Machine

# 4. I2C Transaction State Machine

A typical master transaction state machine:

- IDLE
- START
- SEND_ADDRESS
- WRITE_DATA
- READ_DATA
- REPEATED_START
- STOP
- COMPLETE
- ERROR

This state machine runs inside the ISR and task handler.



# 5. TWI Interrupt Basics on AVR DA

# 5. TWI Interrupt Basics on AVR DA

The TWI master uses:

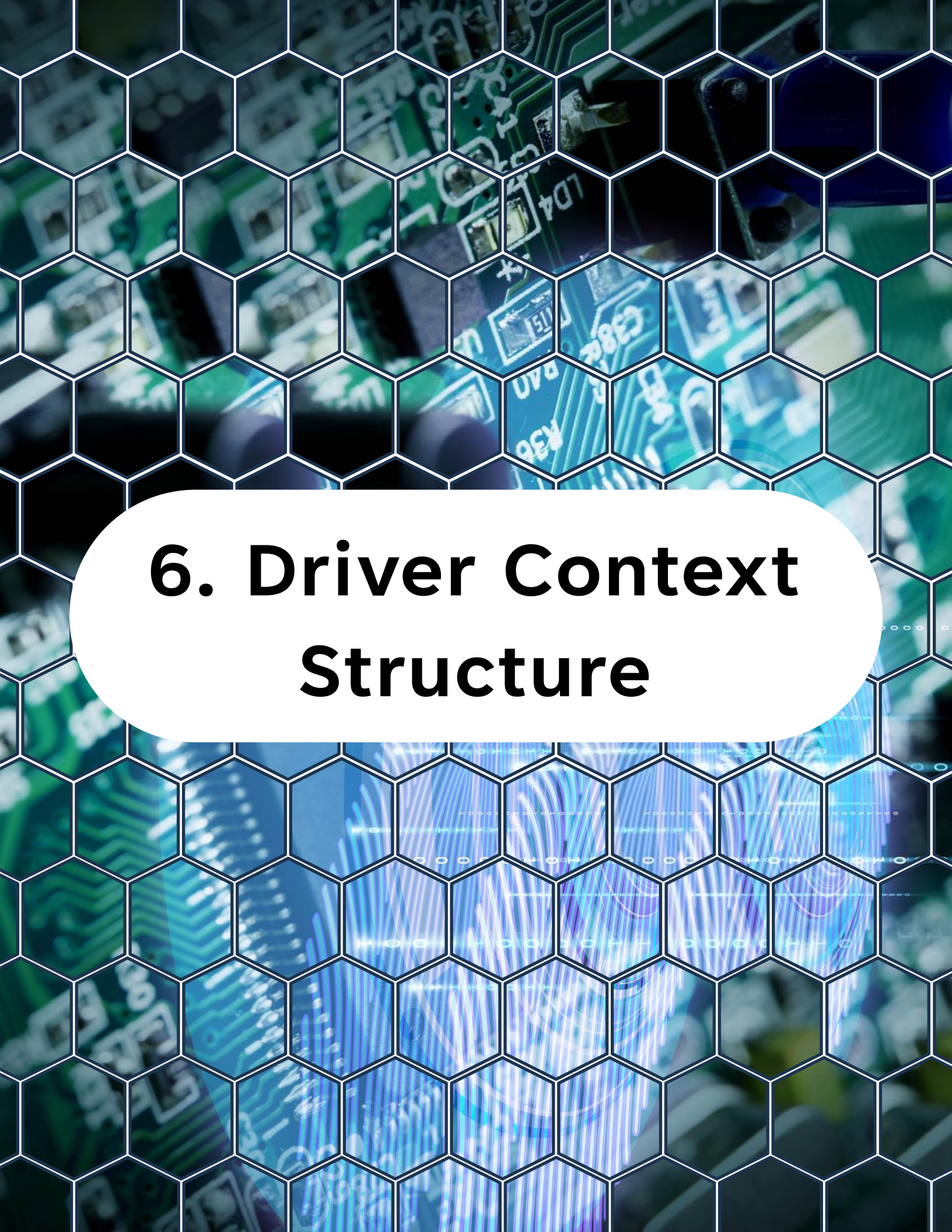
- **WIF** (Write Interrupt Flag)
- **RIF** (Read Interrupt Flag)
- **RXACK** (Acknowledgment status)
- **BUSERR** (Bus Error)
- **ARBLOST** (Arbitration Lost)

Interrupts are enabled via:

```
TWI0.MCTRLA |= TWI_WIEN_bm | TWI_RIEN_bm
```

Your ISR must:

- Identify interrupt source
- Advance state machine
- Handle errors immediately



# 6. Driver Context Structure

# 6. Driver Context Structure

A structured driver requires a context.

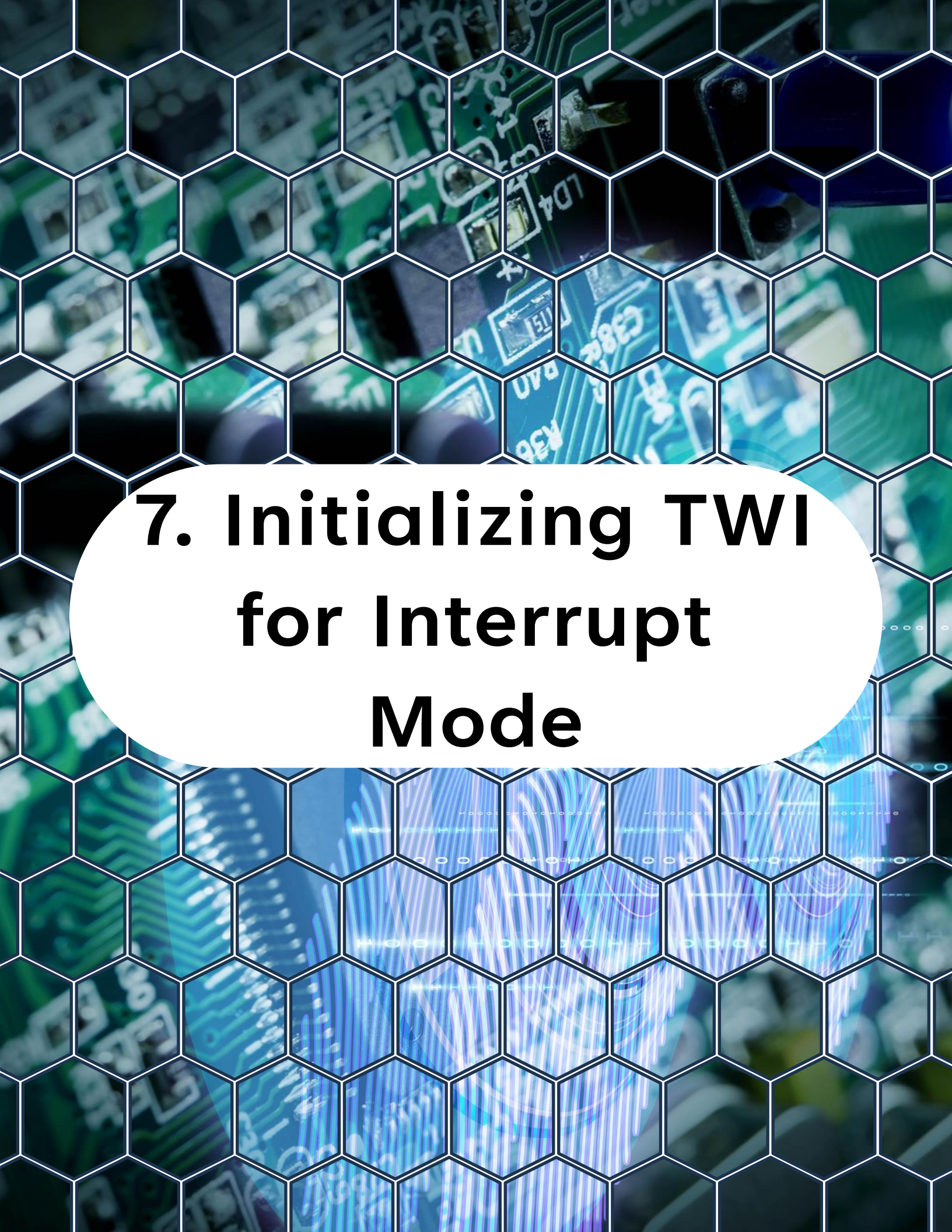
Example:

```
typedef enum
{
    TWI_STATE_IDLE,
    TWI_STATE_START,
    TWI_STATE_WRITE,
    TWI_STATE_READ,
    TWI_STATE_STOP,
    TWI_STATE_COMPLETE,
    TWI_STATE_ERROR
} twi_state_t;

typedef struct
{
    volatile twi_state_t state;

    uint8_t address;
    const uint8_t *tx_buf;
    uint8_t *rx_buf;
    uint16_t tx_len;
    uint16_t rx_len;
    uint16_t tx_index;
    uint16_t rx_index;
```

# 6. Driver Context Structure

```
12 typedef struct
13 {
14     volatile twi_state_t state;
15
16     uint8_t address;
17     const uint8_t *tx_buf;
18     uint8_t *rx_buf;
19     uint16_t tx_len;
20     uint16_t rx_len;
21     uint16_t tx_index;
22     uint16_t rx_index;
23
24     uint8_t error;
25 } twi_ctx_t;
26
27 static twi_ctx_t g_twi;
```

This is your transaction engine.

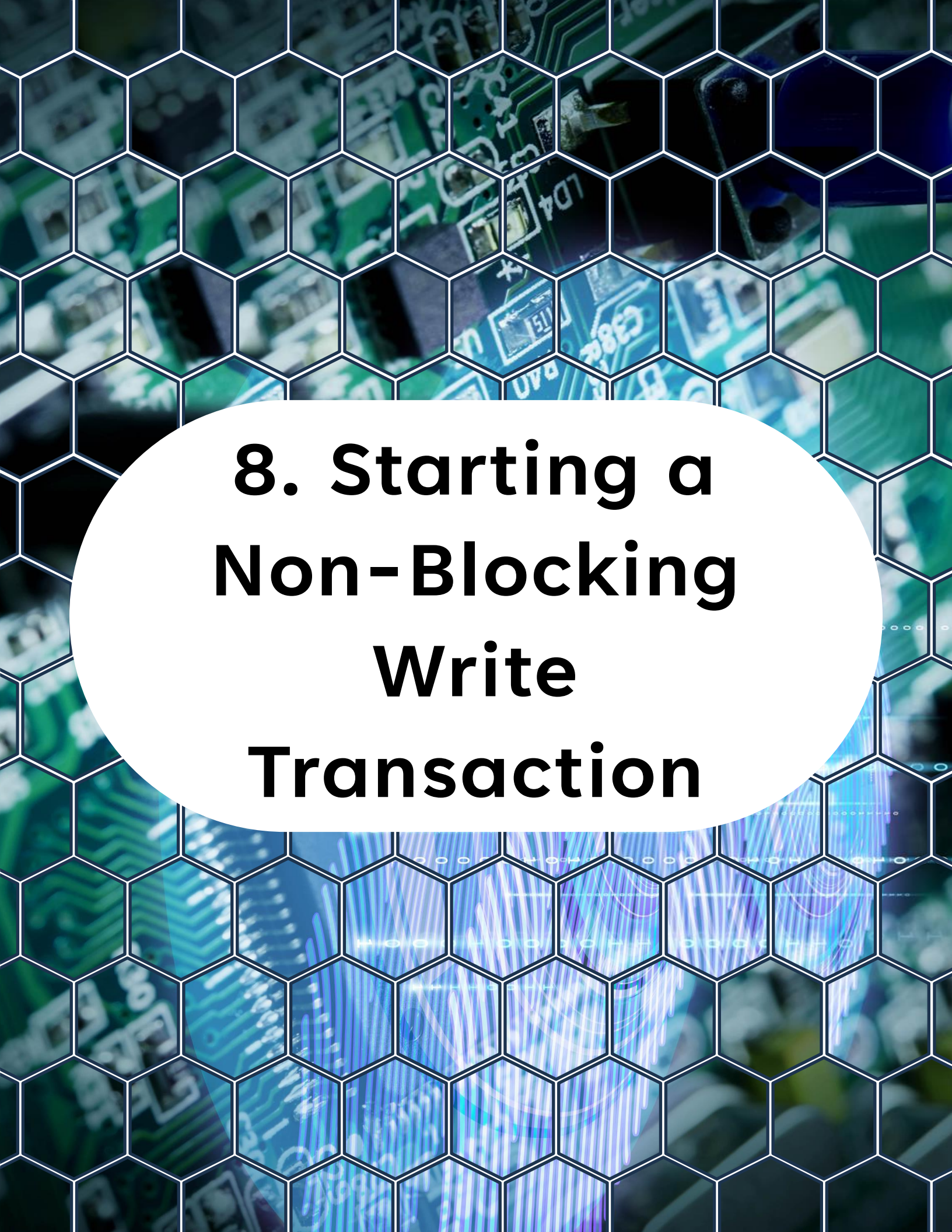


# 7. Initializing TWI for Interrupt Mode

# 7. Initializing TWI for Interrupt Mode

```c
static void twi0_init_interrupt(void)
{
    // Example for 100 kHz at 24 MHz
    TWI0.MBAUD = 75;

    // Enable TWI0 with interrupts for write
    // and read operations
    TWI0.MCTRLA = TWI_ENABLE_bm
                | TWI_WIEN_bm
                | TWI_RIEN_bm;

    // Set bus state to idle
    TWI0.MSTATUS = TWI_BUSSTATE_IDLE_gc;

    g_twi.state = TWI_STATE_IDLE;
}
```

Now TWI events trigger ISR instead of blocking loops.


8. Starting a Non-Blocking Write Transaction

# 8. Starting a Non-Blocking Write Transaction

```c
void twi0_start_write(uint8_t addr,
                      const uint8_t *data,
                      uint16_t length)
{
    if (g_twi.state != TWI_STATE_IDLE)
        return;

    g_twi.address = addr;
    g_twi.tx_buf = data;
    g_twi.tx_len = length;
    g_twi.tx_index = 0;

    g_twi.state = TWI_STATE_START;

    TWI0.MADDR = (addr << 1); /* Write */
}
```

This triggers a START automatically.

The rest happens inside ISR.

# 9. TWI ISR – Write Handling

# 9. TWI ISR - Write Handling

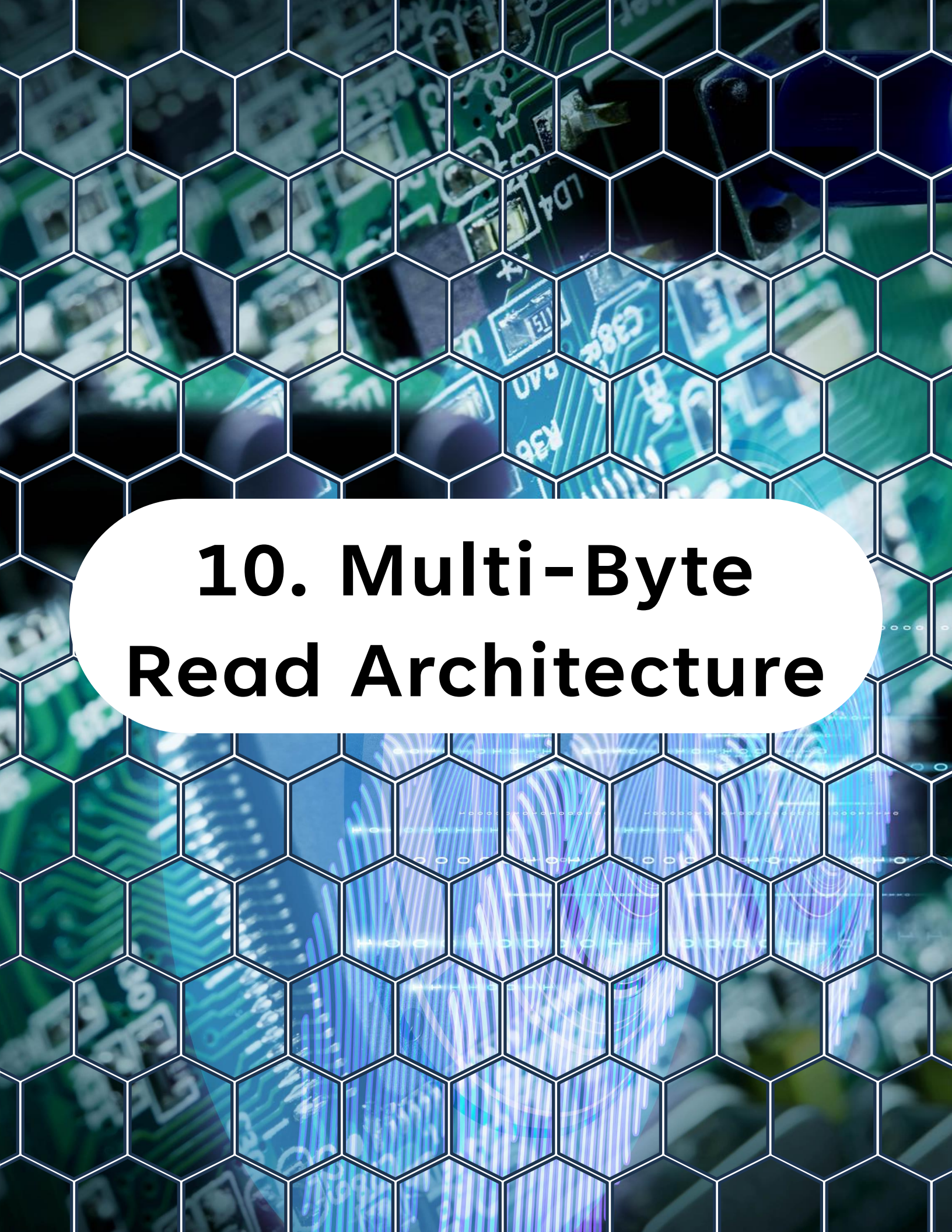
```
// TWI Master Interrupt Service Routine
ISR(TWI0_TWIM_vect)
{
    // Read the status register to clear the interrupt flag
    uint8_t status = TWI0.MSTATUS;

    // Check if the interrupt was caused by
    // a write operation (master transmitting data)
    if (status & TWI_WIF_bm)
    {
        // Check for NACK from the slave
        if (status & TWI_RXACK_bm)
        {
            // Slave did not acknowledge the byte,
            // stop transmission
            g_twi.state = TWI_STATE_ERROR;
            TWI0.MCTRLB = TWI_MCMD_STOP_gc;
            return;
        }

        // Send the next byte of data
        if (g_twi.tx_index < g_twi.tx_len)
        {
            TWI0.MDATA = g_twi.tx_buf[g_twi.tx_index++];
        }
        // If all bytes have been sent, send a STOP condition
        else
        {
            TWI0.MCTRLB = TWI_MCMD_STOP_gc;
            g_twi.state = TWI_STATE_COMPLETE;
        }
    }
}
```

# 9. TWI ISR - Write Handling

The ISR handles:

- ACK checking
- Sending next byte
- Ending transaction

Main loop is never blocked.



# 10. Multi-Byte Read Architecture

# 10. Multi-Byte Read Architecture

```
// TWI Master Interrupt Service Routine
ISR(TWI0_TWIM_vect)
{
    // Read the TWI status register to determine
    // the cause of the interrupt
    uint8_t status = TWI0.MSTATUS;

    // Check if the interrupt was caused by a received byte
    if (status & TWI_RIF_bm)
    {
        // Read received byte and store in buffer
        g_twi.rx_buf[g_twi.rx_index++] = TWI0.MDATA;

        // Check if we need to receive more bytes
        if (g_twi.rx_index < g_twi.rx_len - 1)
        {
            // More bytes to receive, send ACK and
            //continue receiving
            TWI0.MCTRLB = TWI_MCMD_RECVTRANS_gc;
        }
        // Last byte to receive, send NACK and stop condition
        else
        {
            TWI0.MCTRLB = TWI_ACKACT_bm | TWI_MCMD_STOP_gc;
            g_twi.state = TWI_STATE_COMPLETE;
        }
    }
}
```

# 10. Multi-Byte Read Architecture

The ISR handles:

- Send address (read)
- Receive byte
- Send ACK (if more bytes)
- Send NACK + STOP on last byte

# 11. Repeated START Handling

# 11. Repeated START Handling

Many devices require:

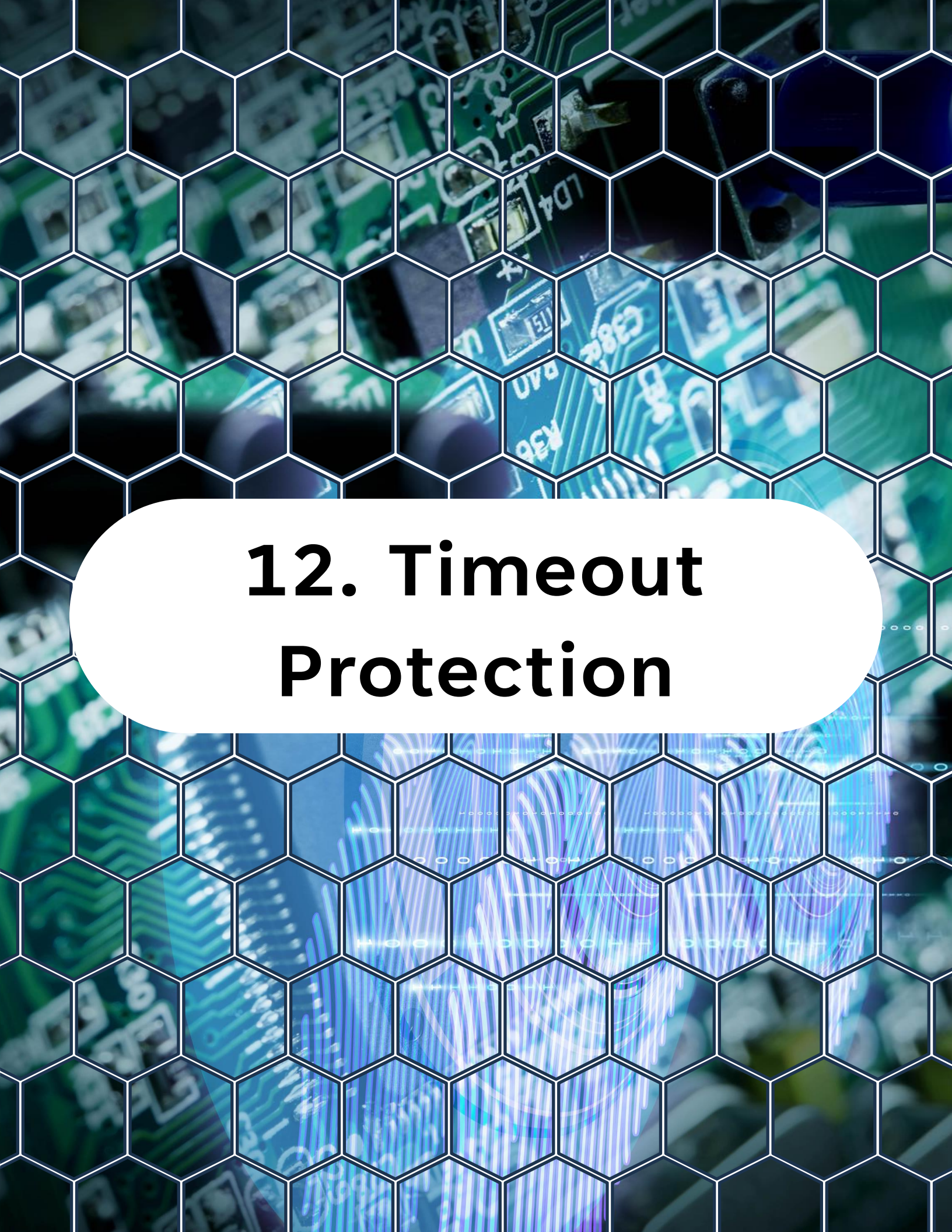
- Write register address
- Repeated START
- Read data

State flow:

- WRITE_PHASE
- REPEATED_START
- READ_PHASE

Never send STOP between register write and read unless device requires it.

Repeated START is critical.



# 12. Timeout Protection

# 12. Timeout Protection

Even interrupt-driven systems can hang.

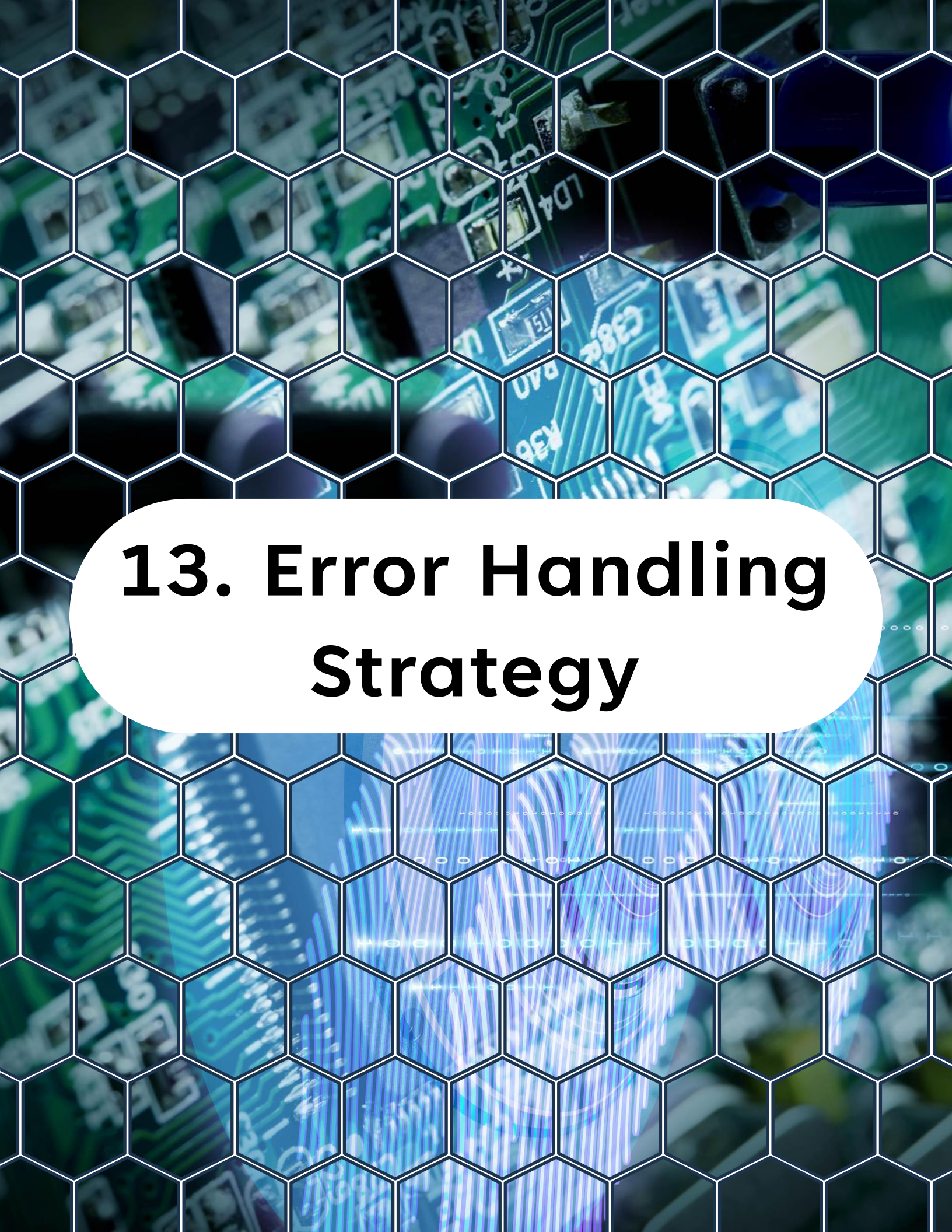
Add timeout using system tick:

```
// If the timeout has been reached, set the
// state to error and send a stop condition.
if ((current_ms - start_ms) > timeout_ms)
{
    g_twi.state = TWI_STATE_ERROR;
    TWI0.MCTRLB = TWI_MCMD_STOP_gc;
}
```

Timeout protects against:

- Stuck bus
- Missing device
- Electrical failure

Never rely on hardware alone.



# 13. Error Handling Strategy

# 13. Error Handling Strategy

Handle immediately:

- RXACK set
- BUSERR
- ARBLOST

Example:

```
// Read the TWI status register
uint8_t status = TWI0.MSTATUS;

// If the TWI bus error bit is set, we have an error
if (status & TWI_BUSERR_bm)
{
    g_twi.state = TWI_STATE_ERROR;
}
else if (status & TWI_ARBLOST_bm)
{
    g_twi.state = TWI_STATE_ERROR;
}
else if (status & TWI_RXACK_bm)
{
    g_twi.state = TWI_STATE_ERROR;
}
else
{
```

# 13. Error Handling Strategy

```
4  // If the TWI bus error bit is set, we have an error
5  if (status & TWI_BUSERR_bm)
6  {
7      g_twi.state = TWI_STATE_ERROR;
8  }
9  else if (status & TWI_ARBLOST_bm)
10 {
11     g_twi.state = TWI_STATE_ERROR;
12 }
13 else if (status & TWI_RXACK_bm)
14 {
15     g_twi.state = TWI_STATE_ERROR;
16 }
17 else
18 {
19     g_twi.state = TWI_STATE_SUCCESS;
20 }
```

Always:

- Force STOP
- Return to IDLE
- Reset internal state

Robust drivers recover gracefully.

# 14. Cooperative Main Loop Pattern

# 14. Cooperative Main Loop Pattern

Main loop should:

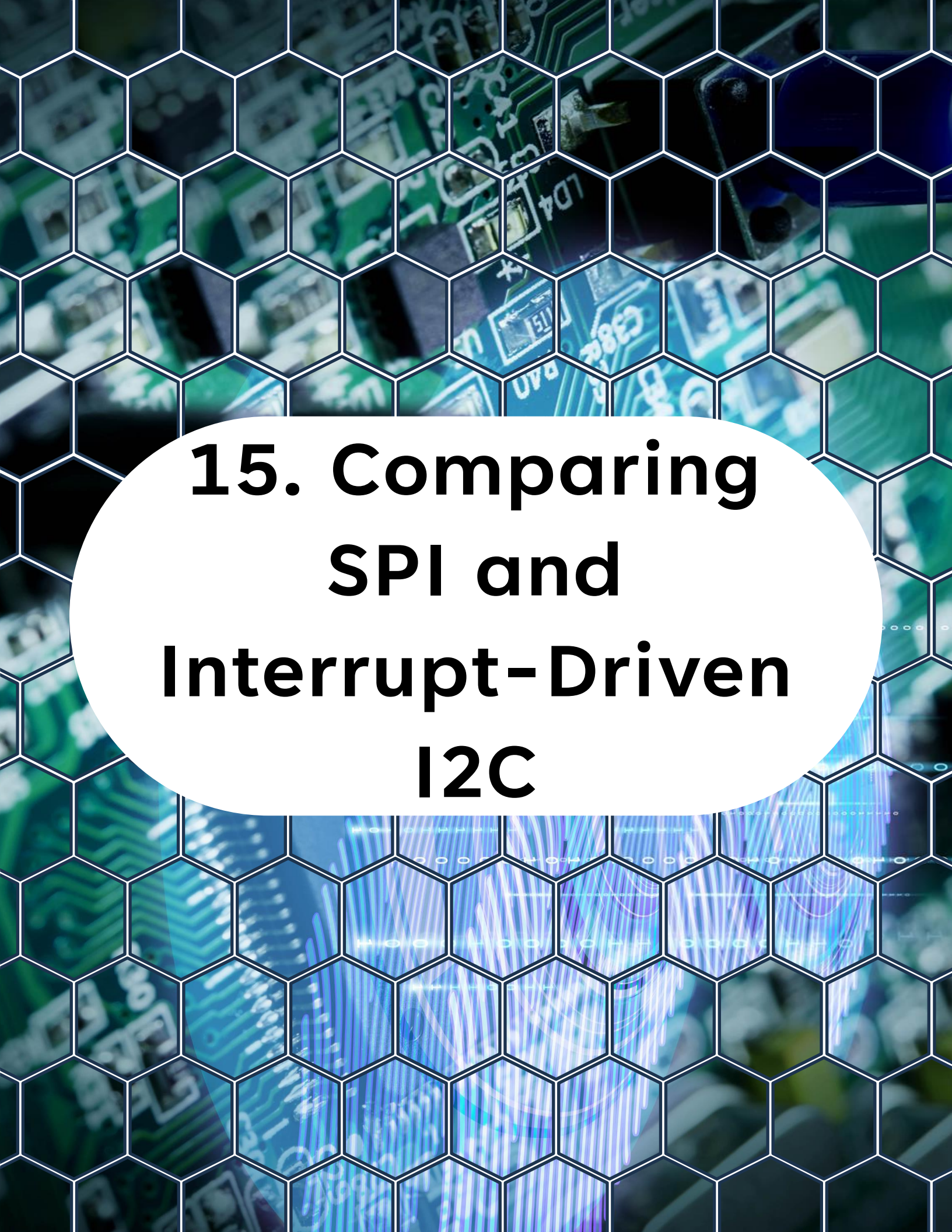
```c
while (1)
{
    twi_task();     /* optional state supervision */
    process_app();  /* rest of firmware */
}
```

Application checks:

```c
if (g_twi.state == TWI_STATE_COMPLETE)
{
    /* Process data */
}
else if (g_twi.state == TWI_STATE_ERROR)
{
    /* Handle error */
}
```

No blocking. No delays.

This is scalable embedded design.



# 15. Comparing SPI and Interrupt-Driven I2C

# 15. Comparing SPI and Interrupt-Driven I2C

SPI ISR:

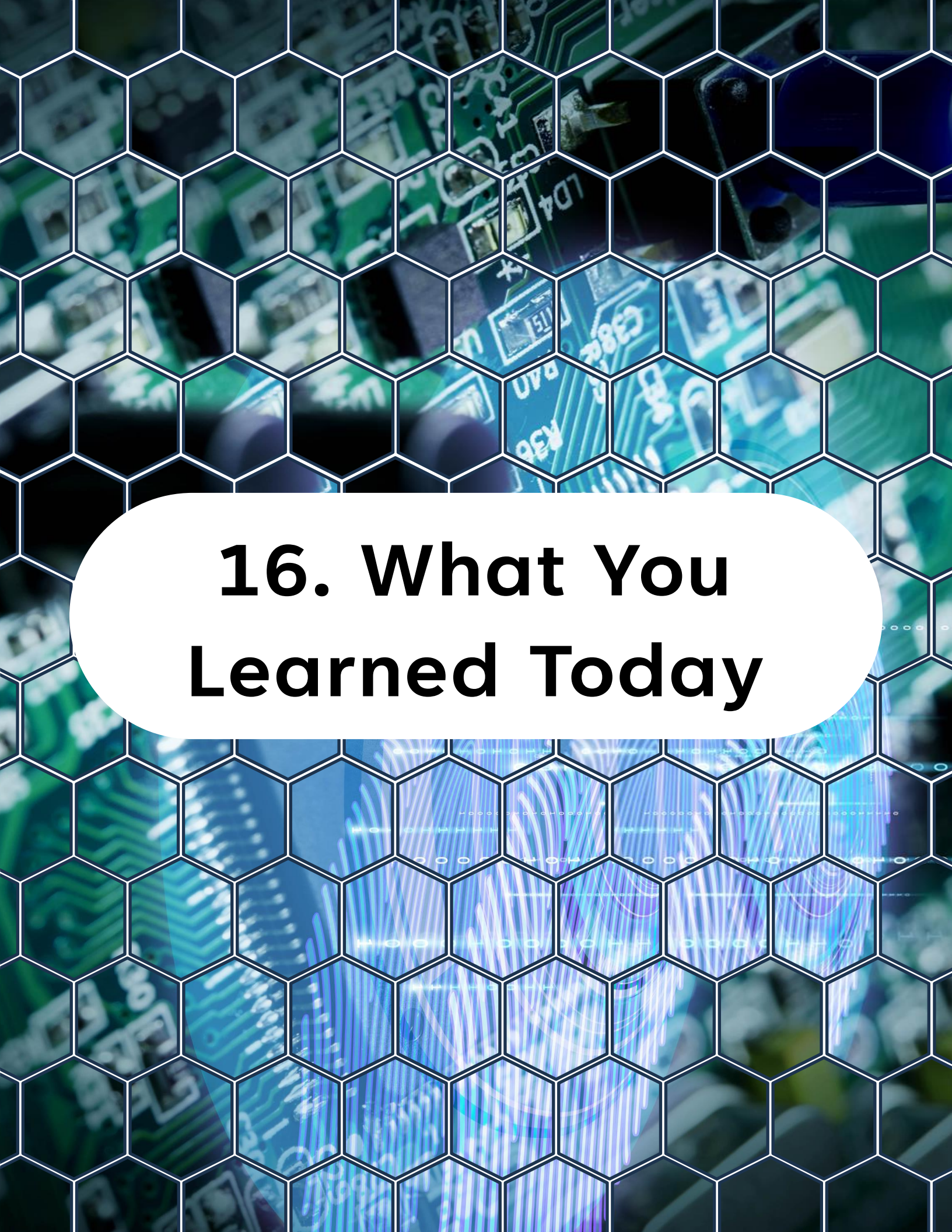
- Byte-by-byte, simple clocking
- No arbitration

I2C ISR:

- Address phase
- ACK/NACK management
- Read vs write flow
- Error states

I2C driver is more complex.

But once structured, it becomes clean and reusable.



# 16. What You Learned Today

# 16. What You Learned Today

By the end of **Day 16**, you understand:

- How to structure a reusable I2C driver
- State machine design for TWI
- Interrupt-driven multi-byte transfers
- Repeated START handling
- Timeout integration
- Error recovery strategies

You are now designing communication stacks like a professional firmware engineer.

# 17. What Comes Next

# 17. What Comes Next

**Day 17** - Analog Comparator (AC) and Event System Integration

Next you will:

- Configure analog comparator
- Use event system to trigger peripherals
- Build hardware-level signal reactions
- Reduce CPU load using event routing

You are moving deeper into real embedded system architecture.

You can find all source code, examples, and updates for this course in the GitHub repository, make sure to visit it, explore the files, and clone it so you can follow along hands-on.

https://github.com/god233012yamil/Bare-Metal-Embedded-Systems-with-AVR128DA48-Course